

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Idea de proyecto
Sistema de Gestión de Donaciones

Trabajo presentado por:

Companocca Checco Roni
Huamani Condori Jeanpiero Sixto

Curso:
Tecnología de objetos

Profesora:
MARIBEL MOLINA BARRIGA

AREQUIPA-PERÚ

2025

SISTEMA INTEGRAL DE GESTIÓN DE DONACIONES CON ENFOQUE EN EFICIENCIA Y TRANSPARENCIA

Qué es SIGDET

Es una plataforma informática diseñada para administrar, registrar y controlar el flujo completo de donaciones dentro de una organización. Su propósito principal es optimizar la gestión de donantes, beneficiarios, inventarios y procesos logísticos, permitiendo una trazabilidad completa desde el origen de la donación hasta su entrega final.

Este sistema integra módulos de backend, frontend e infraestructura para ofrecer una solución centralizada, segura y eficiente, facilitando la transparencia y la toma de decisiones dentro de la institución.

Resultado final del proyecto

El proyecto fue desarrollado por etapas que consta de 5 semanas, la cual se muestra en los videos que se hicieron en cada avance. De igual forma se puede observar en el Github del proyecto.

GitHub del proyecto

<https://github.com/rescobedoq/sigdet>

Video del Avance semana 1

<https://drive.google.com/file/d/1k-090c0jD1xrIzKoOVOSDT8HYWK5S1qa/view?usp=sharing>

Video del Avance semana 2

<https://drive.google.com/file/d/1saLGPHd7ZJeWlqxf6fcJ0jkSVlOeeSD/view?usp=sharing>

Video del Avance semana 3 y 4

<https://www.youtube.com/watch?v=bp2yoYvOiJ0>

Video de la presentación del Proyecto Final

<https://drive.google.com/file/d/18Vqr9bFlvkyXTYLmFiATAjC4MYNgHuUn/view?usp=sharing>

Presentación en Canva material de apoyo

https://www.canva.com/design/DAG6oxoKZSA/PuWG2mkuFdBMerg5ij29Ww/edit?utm_content=DAG6oxoKZSA&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Memoria descriptiva del proyecto

(los datos generales y la introducción, la descripción técnica incluyendo aspectos estructurales)

<https://github.com/rescobedoq/sigdet/blob/jeanpiero-huamani-condori/Memoria%20Descriptiva%20de%20la%20Aplicacion%20SIGDET.pdf>

Manual de usuario

(podemos observar cómo el usuario puede hacer uso de nuestro proyecto)

<https://github.com/rescobedoq/sigdet/blob/jeanpiero-huamani-condori/Manual%20de%20usuario%20SIGDET.pdf>

Introducción

En el contexto de organizaciones no gubernamentales (ONGs) y entidades sociales, la gestión manual de donaciones, registro de donantes, donaciones, inventarios, beneficiarios, distribución suele ser propensa a errores, pérdida de información, duplicaciones, falta de trazabilidad y baja transparencia. Esta situación limita la eficacia operativa de la organización, reduce la confianza de los donantes y dificulta la rendición de cuentas. Como señala un análisis sobre errores comunes en la gestión de donaciones, “la falta de un sistema claro y eficiente para el registro y seguimiento de las donaciones puede llevar a errores en la contabilización, falta de rendición de cuentas y dificultades en la generación de reportes precisos”.

Además, la sostenibilidad de muchas ONGs depende fuertemente de donaciones recurrentes. Pero sin un sistema adecuado, se complica mantener relaciones con donantes, hacer seguimiento, emitir comprobantes o realizar reportes de impacto.

Importancia de un sistema de gestión de donaciones

Un sistema de gestión de donaciones informatizado ofrece múltiples beneficios: centraliza la información de donantes y donaciones, automatiza procesos administrativos, facilita el seguimiento, mejora la eficiencia operativa, permite generar reportes claros y ayuda a mantener la transparencia y confianza con los donantes.

Además, este tipo de sistemas permiten una mejor retención de donantes, a través de comunicaciones más personalizadas, histórico de donaciones, emisión automática de recibos u obras de seguimiento, algo fundamental para la viabilidad a largo plazo de organizaciones sociales.

Por estas razones, diseñar e implementar un sistema como SIGDET robusto, modular, trazable representa una contribución significativa para organizaciones que manejan donaciones, productos, inventarios, beneficiarios y requieren transparencia y control.

Objetivos del Proyecto

Desarrollar un sistema integral de gestión de donaciones (SIGDET) con backend en PostgreSQL y frontend en C++/Qt bajo arquitectura MVC, que permita gestionar de forma eficiente donantes, beneficiarios, donaciones e inventario, garantizando trazabilidad, mantenibilidad, seguridad y facilidad de uso.

Objetivos Específicos

- Implementar una arquitectura modular MVC que separe claramente la lógica de negocio, la interfaz de usuario y el acceso a datos.
- Diseñar e implementar los modelos correspondientes para donantes, beneficiarios, donaciones, inventario y usuarios.
- Construir una interfaz gráfica con Qt Widgets que permita realizar operaciones CRUD, visualización de datos y gestión de inventario/donaciones.
- Integrar la base de datos PostgreSQL mediante Docker para asegurar persistencia, portabilidad y facilidad de despliegue.
- Implementar controladores que manejen la interacción entre la interfaz, la lógica y la base de datos; con validaciones, manejo de errores y control de flujo.
- Permitir la generación de reportes y exportación de datos (por ejemplo, inventario o listados de donaciones/beneficiarios).

- Documentar adecuadamente el sistema: estructura del proyecto, diagramas, guía de compilación y uso, para facilitar mantenimiento y escalabilidad.

Fundamentación Teórica

Arquitectura MVC

En el proyecto SIGDET la arquitectura Modelo-Vista-Controlador (MVC) se implementa dividiendo la aplicación en tres capas bien diferenciadas. La capa Modelo se encarga de la persistencia y estructura de datos, representada por las clases que modelan entidades y por los componentes que gestionan la conexión con PostgreSQL. La capa Vista corresponde a la totalidad de interfaces gráficas desarrolladas con Qt, organizadas en múltiples vistas específicas para cada módulo funcional del sistema. Finalmente, la capa Controlador gestiona la lógica de aplicación, comunicando las vistas con los modelos, validando entradas, procesando solicitudes y asegurando la coherencia del flujo de la aplicación.

Esta división permite una mayor mantenibilidad del código, facilita el escalamiento del sistema y promueve buenas prácticas de ingeniería de software.

Componente	Elementos del Proyecto	Descripción Funcional
Modelo (Model)	• userModel.cpp / .h • donanteModel.cpp / .h • beneficiarioModel.cpp / .h • PostGreDII/ (scripts y estructura de BD) • conexiondbhelperController.cpp / .h • databasemanagerController.cpp / .h	Implementan la gestión de datos y la lógica de negocio. Representan entidades del sistema (usuarios, donantes, beneficiarios, inventario y donaciones). Gestionan la conexión con PostgreSQL, ejecutan consultas, administran transacciones y proporcionan datos consistentes al controlador.
Vista (View)	• loginwindowView.cpp / .h • donacionesrecibidasView.cpp / .h • donacionesenviadasView.cpp / .h • beneficiarioView.cpp / .h • donanteView.cpp / .h • inventarioView.cpp / .h • informesView.cpp / .h • tablewidget.cpp / .h • tablewidgetcardView.cpp / .h • sidebarView.cpp / .h • ayudaView.cpp / .h	Corresponden a la interfaz gráfica desarrollada con Qt. Muestran tablas, formularios, reportes, paneles laterales e íconos. Interactúan con el usuario mediante eventos, capturan entradas y presentan la información procesada por el controlador. No contienen lógica de negocio.

Controlador (Controller)	• iconhelperController.cpp / .h • conexiondbhelperController.cpp / .h (también cumple rol de acceso a datos) • databasemanagerController.cpp / .h • registromodalView.cpp / .h (manejo de lógica de registro) • mainwindow.cpp / .h	Actúan como intermediarios entre vistas y modelos. Gestionan la lógica de interacción: validación de datos ingresados, coordinación de consultas hacia los modelos, control del flujo de ventanas, manejo de sesiones y actualización de las vistas. Orquestan los procesos del sistema para que la interfaz muestra información precisa y actualizada.
-------------------------------------	---	--

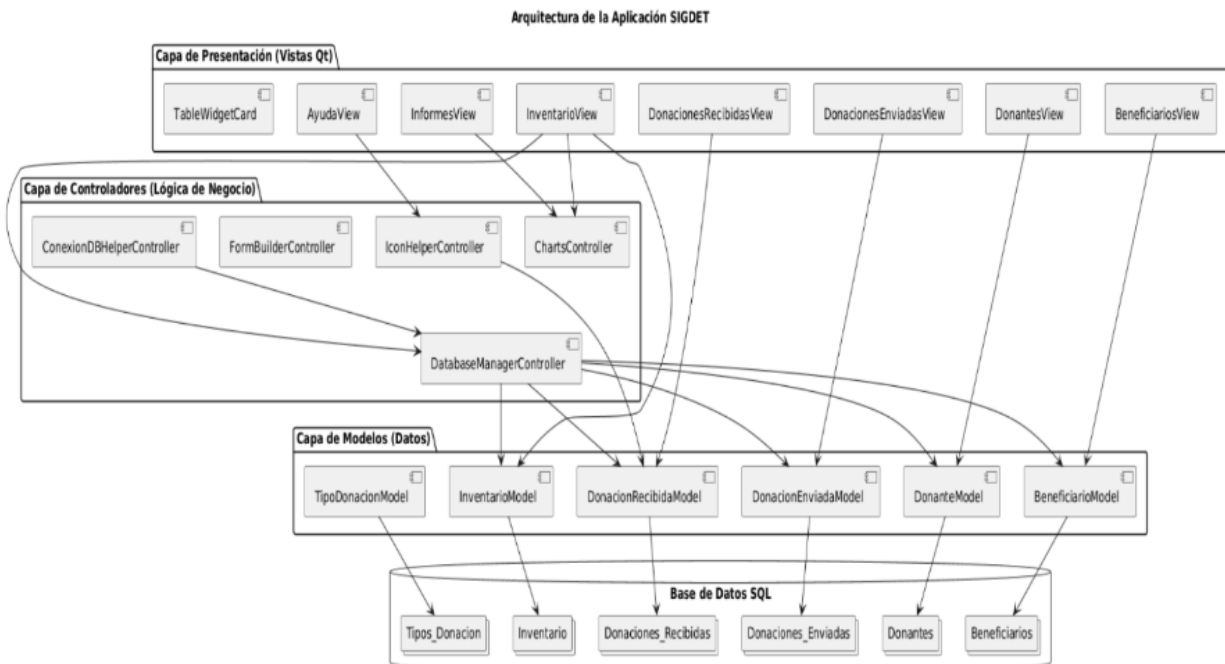
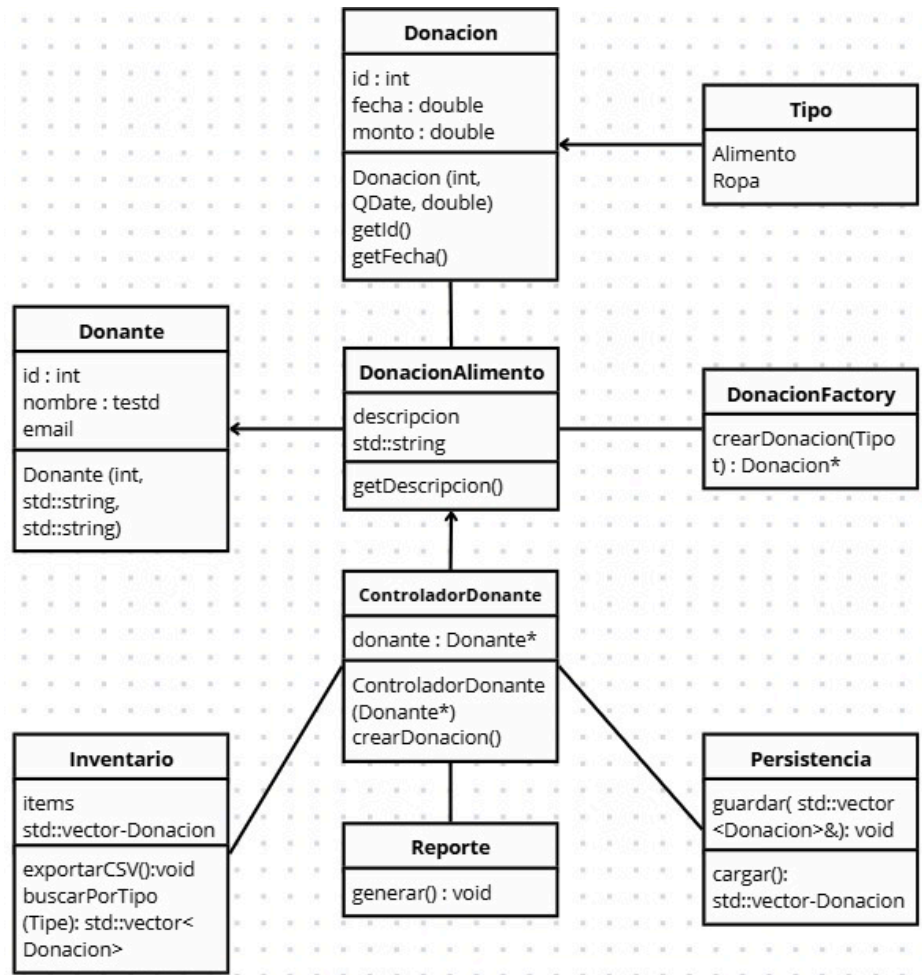


Diagrama UML inicial



Programación Orientada a Objetos

La POO nos permite construir SIGDET como un sistema modular, escalable y fácil de mantener. El uso de clases, herencia, polimorfismo y encapsulamiento favoreció un desarrollo ordenado y extensible, adecuado para un sistema de gestión complejo como la administración de donaciones, usuarios e inventarios.

En SIGDET, cada entidad importante del sistema se representa a través de una **clase**, siguiendo el principio fundamental de la POO:

Ejemplos de clases Modelo en nuestra arquitectura:

- donanteModel.h / .cpp
- beneficiariomodel.h / .cpp
- userModel.h / .cpp

Estas clases encapsulan datos como:

```
C/C++
QString nombre;
QString dni;
QString direccion;
QString correo;
```

y exponen métodos públicos para gestionarlos

```
C/C++
QString getNombre() const;
void setNombre(const QString &value);
```

Encapsulamiento aplicado a la seguridad y consistencia de datos

Casi todos nuestros modelos siguen la estructura:

- Atributos privados
- Métodos públicos de acceso y modificación

Esto garantiza:

- Protección de datos sensibles
- Acceso controlado
- Prevención de errores en la lógica de negocio

Por ejemplo, en userModel, los datos del usuario nunca son manipulados directamente desde la vista.

Abstracción como forma de simplificar el sistema

Cada clase del proyecto encapsula únicamente los datos relevantes para su función:

- DonanteModel: datos de donantes
- BeneficiarioModel: datos de beneficiarios
- UserModel: credenciales de acceso
- InventarioView: representación visual del inventario

Esto elimina detalles innecesarios y simplifica la complejidad del sistema, lo que es justamente la abstracción.

```
donanteModel::donanteModel(int id, const QString &nombre, const QString &apellido,
                             const QString &email, const QString &telefono,
                             const QString &direccion, const QString &tipo_documento,
                             const QString &numero_documento,
                             const QDateTime &created_at, const QDateTime &updated_at)
: id(id), nombre(nombre), apellido(apellido), email(email),
  telefono(telefono), direccion(direccion),
  tipo_documento(tipo_documento), numero_documento(numero_documento),
  created_at(created_at), updated_at(updated_at) {}
```

Herencia y reutilización en componentes visuales (Views)

En la parte visual del frontend, varias clases heredan funcionalidad de Qt:

- beneficiarioView hereda de QWidget
- donanteView hereda de QWidget
- loginwindowView hereda de QMainWindow

- mainWindow hereda de QMainWindow
- sidebarView hereda de QWidget
- tablewidget hereda de QTableWidgetItem

Este uso de herencia de clases Qt permite:

- reutilizar componentes
- extender funcionalidades
- mantener alto nivel de modularidad

Por ejemplo, tablewidgetcardView extiende un card visual del inventario usando herencia.

```
QJsonArray DonacionesRecibidasView::getDonacionesRecibidas()
{
    DatabaseManager db;
    QString queryStr = R"(
        SELECT
            id,
            donante_id,
            descripcion,
            cantidad,
            unidad,
            estado,
            TO_CHAR(fecha_donacion, 'YYYY-MM-DD') AS fecha_donacion
        FROM donaciones_recibidas
        ORDER BY cantidad;
    )";

    return db.doGet(queryStr);
}

QWidget* DonacionesRecibidasView::crearResumenCards(const QJsonArray &data) {
    auto *container = new QWidget;
    auto *layout = new QHBoxLayout(container);
    layout->setSpacing(16);
    layout->setContentsMargins(0, 0, 0, 0);

    // * Calcular estadísticas
    int totalCantidad = 0;
    int pendientes = 0;
    int recibidasHoy = 0;

    QString hoy = QDate::currentDate().toString("yyyy-MM-dd");
```

Polimorfismo en el manejo de señales, ranuras y controladores

Los controladores de SIGDET:

- conexiondbhelperController
- databasemanagerController
- iconhelperController

aprovechan el polimorfismo propio de Qt a través de:

- Señales (signals)
- Ranuras (slots)
- Métodos virtuales de QWidget y QMainWindow

Para el desarrollo del sistema se utilizó Qt Framework para la creación de la interfaz gráfica y el manejo de eventos; PostgreSQL como base de datos relacional que garantiza integridad y rendimiento; y Docker para la contenerización del entorno, facilitando la portabilidad y despliegue. En el código C++, se emplearon punteros inteligentes para asegurar una gestión de memoria automática y segura. Además, Git y GitHub se utilizaron para el control de versiones, permitiendo documentar el progreso, mantener un historial de cambios y asegurar la colaboración eficiente durante el proyecto.


```

#include "databaseManagerController.h"

DatabaseManager::DatabaseManager(QObject *parent)
    : QObject(parent)
{
    // Crear la conexión con nombre único (si no existe)
    if (!QSqlDatabase::contains("donaciones_conn")) {
        db = QSqlDatabase::addDatabase("QPSQL", "donaciones_conn");
        db.setHostName("127.0.0.1");
        db.setPort(5432);
        db.setDatabaseName("donaciones_db");
        db.setUserName("donaciones_user");
        db.setPassword("donaciones_pass");
    } else {
        db = QSqlDatabase::database("donaciones_conn");
    }
}

bool DatabaseManager::open() {
    if (!db.isOpen()) {
        if (!db.open()) {
            qWarning() << "❌ Error al conectar a la BD:" << db.lastError().text();
            return false;
        }
        qDebug() << "✅ Conexión establecida con la base de datos";
    }
    return true;
}

```

Desarrollo del Proyecto

Semana 1 – Análisis, Diseño y Base del Proyecto

Objetivo: Establecer la estructura base del proyecto y definir el modelo de clases.

Tareas:

- Configuración del entorno (Qt Creator/CMake/Git).
- Creación del repositorio y estructura inicial (src/, include/, data/, tests/).
- Definición de clases principales del Modelo (Donante, Donación, Inventario, Reporte) .
- Elaboración de un diagrama UML inicial.
- Configuración del archivo CMakeLists.txt .
- Implementación de un main básico que compila correctamente.
- Documentación inicial en el README.md.

Entrega esperada: Proyecto compila correctamente con estructura MVC vacía y modelos definidos.

Semana 2 – Implementación del Modelo (Lógica de Negocio)

Objetivo: Desarrollar toda la lógica de negocio del sistema sin interfaz gráfica.

Tareas:

- Implementación de las clases: Donante, Donacion, Inventario, Reporte.
- Aplicar herencia y polimorfismo (DonacionAlimento, DonacionRopa).
- Implementar Factory Method para creación de objetos de donación.
- Agregar excepciones personalizadas para control de errores.
- Pruebas iniciales de CRUD desde consola.
- Implementar pruebas unitarias básicas (GoogleTest o Catch2). Entrega esperada: Lógica del sistema completamente funcional en modo consola.

Semana 3 – Interfaz Gráfica (Vista) y Controladores

Objetivo: Implementar la interfaz Qt y conectar los controladores con el modelo.

Tareas:

- Creación de ventanas con Qt Widgets :Ventana principal.
- Formulario de Donantes y Donaciones. Tabla de Inventario.
- Implementar controladores: ControladorDonante, ControladorDonacion.
- Conectar señales y ranuras (MVC).
- Integrar validaciones visuales y mensajes de error (QMessageBox).
- Prueba completa del CRUD desde la interfaz. Entrega esperada: Aplicación Qt funcional con operaciones CRUD completas sobre estructuras en memoria.

Semana 4 – Persistencia y Funcionalidades Avanzadas

Objetivo: Incorporar almacenamiento persistente y optimizar el rendimiento.

Tareas:

- Implementar persistencia de datos:
- SQLite (QSqlDatabase) o JSON (QJsonDocument/ nlohmann::json).
- Exportación de datos (Inventario / Reportes) a CSV o PDF .
- Implementar búsqueda y filtrado de donaciones por tipo, fecha o cantidad.
- Uso de subprocesos múltiples (QThread) para cargas masivas o procesos lentos.
- Implementar Observer para actualizar la vista en tiempo real.
- Mejoras visuales y pulido de interfaz. Entrega esperada: Aplicación persistente, con informes exportables y GUI estable.

Semana 5 – Pruebas, Optimización y Documentación Final

Objetivo: Finalizar, probar y documentar completamente el proyecto.

Tareas:

- Pruebas unitarias y de integración completas.
- Revisión de excepciones, validaciones y manejo de errores.
- Optimización de rendimiento y limpieza de código.
- Documentación final en el README.md:
- Descripción general.
- Diagrama UML actualizado.
- Capturas o GIF de la interfaz Qt.
- Guía de compilación y ejecución con CMake .

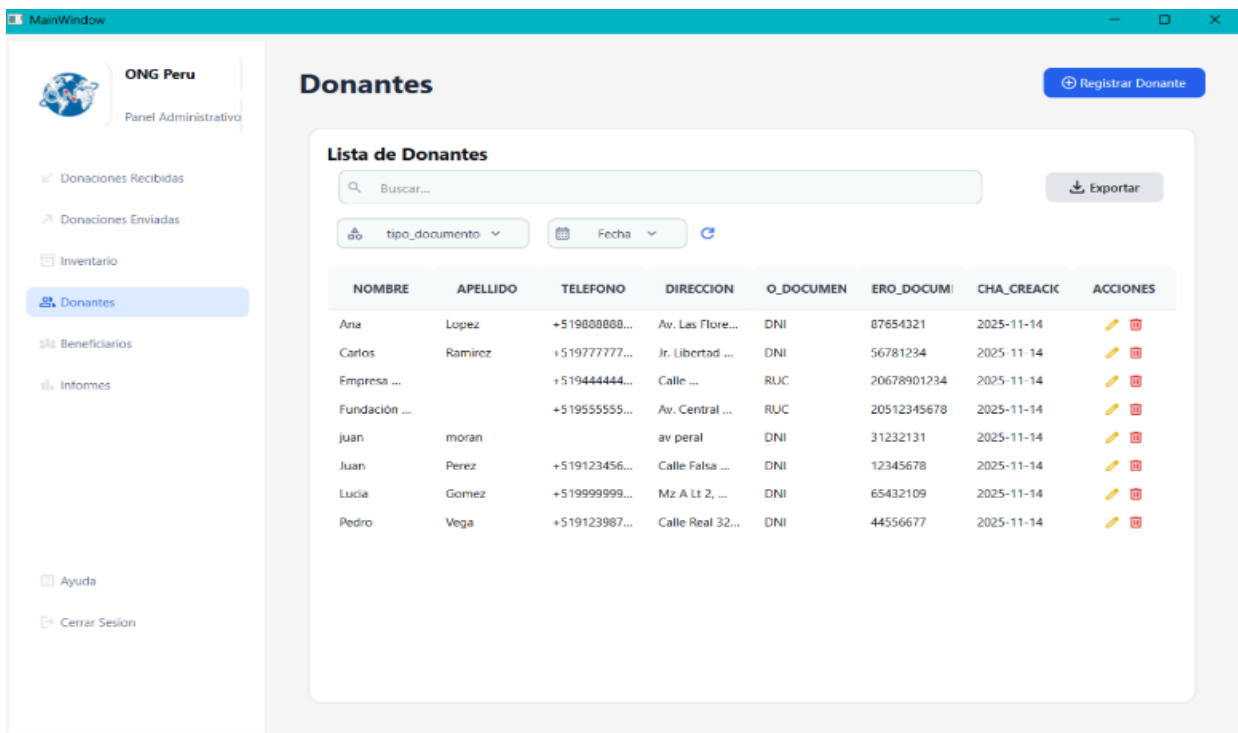
Presentación final y entrega del ejecutable. Entrega esperada: Sistema completo, probado y documentado, listo para presentación.

Resultados

Al iniciar la aplicación, se mostrará el formulario de inicio de sesión. El usuario debe ingresar:



Gestión de donantes



NOMBRE	APELLIDO	TELEFONO	DIRECCION	O_DOCUMEN	ERO_DOCUMI	CHA_CREACI	ACCIONES
Ana	Lopez	+519888888...	Av. Las Flore...	DNI	87654321	2025-11-14	 
Carlos	Ramirez	+519777777...	Jr. Libertad ...	DNI	56781234	2025-11-14	 
Empresa ...		+519444444...	Calle ...	RUC	20678901234	2025-11-14	 
Fundación ...		+519555555...	Av. Central ...	RUC	20512345678	2025-11-14	 
Juan	moran		av peral	DNI	31232131	2025-11-14	 
Juan	Perez	+519123456...	Calle Falsa ...	DNI	12345678	2025-11-14	 
Lucia	Gomez	+519999999...	Mz A Lt 2, ...	DNI	65432109	2025-11-14	 
Pedro	Vega	+519123987...	Calle Real 32...	DNI	44556677	2025-11-14	 

Al administrar donantes

- Hacer clic en el módulo donantes.
- Seleccionar Agregar para registrar uno nuevo

Registrar en donantes

Registrar nuevo registro en 'donantes'

nombre:

juan

apellido:

morales

email:

morales@unsa.edu.pe

telefono:

123213412

direccion:

quipa urb rosas 232

tipo_documento:

DNI

numero_documento:

720909340

Guardar

Gestión de beneficiarios

Beneficiarios

Registrar Beneficiario

Lista de Beneficiarios

Buscar...

Exportar

referencia

Fecha

NOMBRE	APELLIDO	REFERENCIA	DIRECCION	TELEFONO	ECHA_CREACIOI	ACCIONES
Centro Médico ...		Atención médi...	Av. Salaverry ...	+5195555555	2025-04-14	 
Colegio San ...		Institución ...	Jr. Los Héroes ...	+5194444444	2025-05-14	 
Hogar San Juan		Hogar de niños	Calle Esperanza...	+51910101010	2025-09-14	 
Luis	Molina	Familia en ...	Calle 9 de ...	+5193333333	2025-06-14	 
María	Gonzales	Centro ...	(Av. Principal 45...)	+51987654321	2025-10-14	 
niños del mare		hogar de niños			2025-11-14	 

Gestión de donaciones recibidas

Donaciones Recibidas

Registrar Donación

Total Donaciones Recibidas

345

Desde el inicio de operaciones

Donaciones Pendientes

0

Requieren procesamiento inmediato

Donaciones Recibidas Hoy

0

Actualizado hace unos minutos

Detalle de Donaciones Recibidas

Buscar...

Exportar

donante_id

Fecha

DONANTE_ID	DESCRIPCION	CANTIDAD	UNIDAD	ESTADO	FECHA DONACION	ACCIONES
3	ropa	10	unidades	registrada	2025-12-03	
4	Arroz kg	25	kg	registrada	2025-11-09	
7	Medicinas ...	40	cajas	registrada	2025-11-12	
3	Kits de limpieza	40	paquetes	registrada	2025-11-08	
1	Fideos 1kg	50	kg	registrada	2025-11-04	
5	Juguetes varios	60	unidades	registrada	2025-11-10	
6	Útiles escolares	120	unidades	registrada	2025-11-11	

Gestión de donaciones enviadas

Donaciones Enviadas

Registrar Envío

Total Envíos Realizados

66

Desde el inicio de operaciones

Envíos en Tránsito

0

Con entrega programada pronto

Envíos Entregados Hoy

0

Actualizado hace unos minutos

Detalle de Donaciones Enviadas

Buscar...

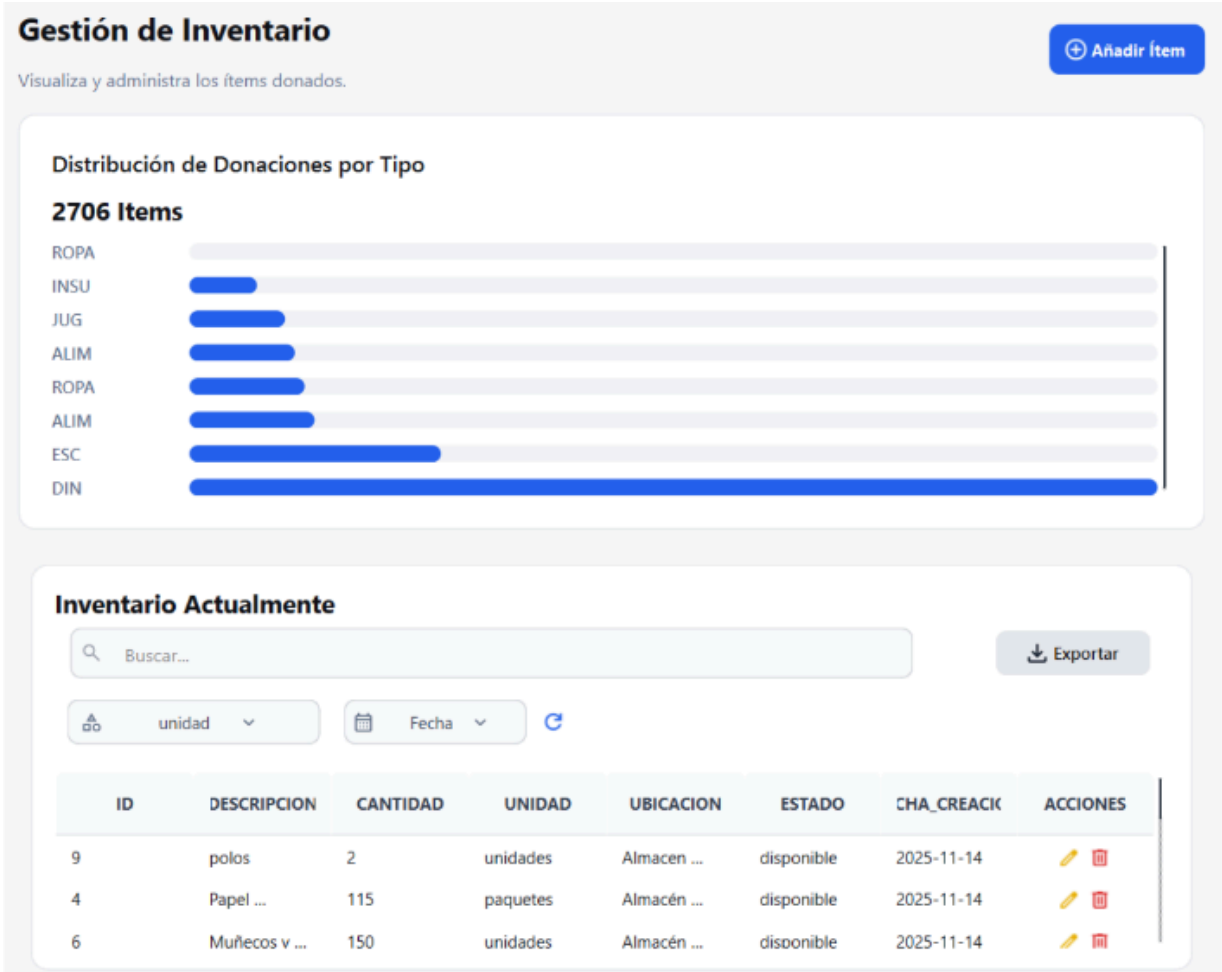
Exportar

beneficiario_id

Fecha

BENEFICIARIO_ID	DESCRIPCION	CANTIDAD	UNIDAD	ESTADO	FECHA_ENVIO	ACCIONES
0	Kits de limpieza	5	paquetes	enviado	2025-11-13	
2	Abrigos de ...	10	unidades	enviado	2025-11-13	
0	Medicamentos ...	10	cajas	enviado	2025-11-13	
1	envío de arroz	11	kg		2025-11-14	
5	Útiles escolares...	30	unidades	enviado	2025-11-11	

Gestión de inventario



Conclusiones

El desarrollo del sistema SIGDET permitió implementar una arquitectura MVC sólida que organizó adecuadamente la interfaz, la lógica de negocio y la gestión de datos, logrando módulos funcionales para usuarios, donantes, beneficiarios, inventario y donaciones integrados con una base de datos PostgreSQL y una interfaz gráfica construida en Qt, todo ello soportado por Docker y Git para asegurar portabilidad y control de versiones. Durante el proyecto se aprendió a aplicar de manera práctica principios de programación orientada a objetos, manejo de consultas y transacciones, diseño de interfaces con signals y slots, despliegue de contenedores y flujo profesional de desarrollo. Técnicamente, el sistema representa una solución robusta, modular y escalable, alineada con estándares actuales de ingeniería de software. Finalmente, se identifican mejoras relevantes como fortalecer la seguridad mediante roles, optimizar consultas, agregar validaciones más estrictas, incluir pruebas automatizadas y ampliar las capacidades de integración mediante una API REST y reportes avanzados, con el fin de incrementar el rendimiento y la usabilidad del sistema.

Recomendaciones

Lo que se recomienda es que en futuras optimizaciones del sistema SIGDET, se realice un ajuste más profundo de las consultas SQL para mejorar el rendimiento en módulos con alta carga de datos y se

implemente un sistema de cacheo que reduzca accesos repetitivos a la base de datos. Asimismo, sería beneficioso incorporar funciones adicionales como una API REST para permitir la integración con aplicaciones móviles o plataformas externas, así como módulos de reportes automáticos y notificaciones que agilicen la gestión operativa.

En cuanto a la experiencia de usuario, se aconseja mejorar la navegabilidad mediante paginación en tablas, estandarización de estilos, retroalimentación visual más clara y procesos más intuitivos en el registro de información.

Bibliografía

1. Casals Ordóñez, Francisco Javier. *Diseño e Implementación de Sistemas de Gestión de Donaciones y Administración de Información de Internos de la Fundación Niño y Patria*. Tesis. 2010. Esta tesis documenta cómo una fundación implementa un sistema de gestión de donaciones, acopio y despacho de bienes. [Repositorio Académico](#)
2. “NGO Donor Management System | Donation Tracking & Fundraising CRM – Zama254”. Blog. Describe los beneficios de un sistema de gestión de donantes: centralización, transparencia, automatización, eficiencia. [Zama Web Experts](#)
3. “Comprehensive Analysis of Donation Tracking Software”. Artículo técnico/sensible. Resalta cómo un software de rastreo de donaciones mejora la supervisión financiera, eficiencia operativa, reportes y reduce errores humanos. [Bi2Dev](#)
4. Planfix Blog: “Why non-profits need CRM systems”. Explica cómo un CRM para ONGs mejora la retención de donantes, personalización de comunicaciones, automatización de tareas administrativas y eficiencia general. [planfix.com](#)
5. “How NGOs Can Use Automation to Save Time on Donor Management” — artículo en fundsforngos.org. Aborda cómo la automatización en la gestión de donaciones reduce carga administrativa y mejora la precisión. [www2.fundsforngos.org](#)